[UNIVERSITY]

[Faculty]

Diploma Project Brief

JTAG-Accessible Memory BIST on the AMD AUP ZU3

Student: to be assigned

[Department, e.g. Microelectronics]

Supervisor: Arman Manukyan, PhD

Yerevan, 2026

Version 0.1 · October 2026 · Draft

Topic at a Glance

JTAG-Accessible Memory BIST on the AMD AUP ZU3

Work type	Degree level	Format	Duration	Open places
Bachelor diploma project	**Bachelor**	**Individual (one student)**	**2 semesters (30 weeks)**	**1**

People

Student: to be assigned
[Department, e.g. Microelectronics]
Supervisor: Arman Manukyan, PhD

Timeline

Language of the work
English

Field and requirements

Research areas	Design for testability · Memory testing · Test access (IEEE 1149.1 · IEEE 1687) · Digital design · FPGA · Functional verification
Keywords	Memory BIST · March tests · fault injection · JTAG · IEEE 1687 (IJTAG) · at-speed test · clock-domain crossing · SystemVerilog · SVA
Tools and equipment	AMD AUP ZU3 board (Zynq UltraScale+ MPSoC), Vivado (Simulator, IP Packager, IP Integrator), Vitis
Prerequisites	Digital logic and FSMs, an HDL (Verilog/SystemVerilog), C programming

Contents

1 Context and Motivation . . . 1
2 Problem and Project Idea . . . 2
3 Goals and Expected Outcomes . . . 6
4 Scope, MVP and Constraints . . . 7
5 Memories, Fault Models and Test Algorithms . . . 8
6 BIST Architecture . . . 10
7 Test Access and Clock Modes . . . 12
8 Software Tester and the Fault-Coverage Experiment . . . 14
9 Verification Plan . . . 15
10 Tasks . . . 18
11 Learning Path . . . 19
12 Getting Started . . . 22
13 Work Plan and Milestones . . . 24
14 Deliverables . . . 25
15 Assessment and Acceptance Criteria . . . 25
16 Final Demonstration . . . 27

17 Extensions (Optional) 27
18 Supervision, Collaboration and Risks 28
19 Weekly Progress Tracker 29
References and Reading List 31
List of Abbreviations 33
Glossary 34

1. Context and Motivation

Modern chips contain hundreds or thousands of embedded memories, and memories take a large share of the chip area. Because memory cells are the densest structures on silicon, they are where manufacturing defects most often appear. Every memory must therefore be tested after production, and often again in the field. Chips include **Memory Built-In Self-Test (MBIST)**: on-chip hardware that applies test algorithms to the memories, compares the results and reports pass/fail and the failing locations.

Two further questions decide how such a system works in a real chip:

- **How does the tester reach the BIST?** A chip has only a few test pins. The standard entrance is the **Joint Test Action Group (IEEE 1149.1 test access port) (JTAG)** Test Access Port (TAP) (IEEE 1149.1 [U1]). Behind it, the many embedded instruments (BIST controllers, sensors, configuration registers) are reached through a reconfigurable scan network defined by **IEEE 1687 (Internal JTAG (IEEE 1687) (IJTAG))** [U2, A1]. Commercial memory-BIST solutions are controlled exactly this way.
- **At which clock does the test run?** Some defects (a slow cell, a weak sense amplifier) only show up at the memory's real operating speed. Memories are therefore tested at-speed test on a fast clock, while the test access itself runs on the slow JTAG clock. Moving commands and results safely between these clocks is a Clock-Domain Crossing (CDC) problem.

This project builds such a system by hand, in **SystemVerilog**, as a set of **reusable test IP blocks**: a TAP controller, an IEEE 1687 network, a programmable BIST controller with memory collars, fault-injection wrappers and a clock/mode controller. The blocks are verified with modern verification techniques and assembled into a complete system on an FPGA.

Platform. AMD **AUP ZU3** board (**Zynq UltraScale+ Multiprocessor System-on-Chip (MPSoC)**). All test IPs and memories are in the programmable logic (Programmable Logic (PL)). The ARM processing system (Processing System (PS)) only plays the role of the external *tester*: it drives the JTAG pins of the student's TAP through General-Purpose Input/Output (GPIO), like a JTAG adapter would. Tools: **Vivado** (simulation, IP Packager, IP Integrator) and **Vitis**.

Why fault injection? The block RAMs of a production FPGA essentially never fail, so a BIST run on them would always report "pass". Every memory is therefore wrapped in a *fault-injection wrapper* that makes it behave like a defective memory, including a *speed-dependent* fault that appears only at the fast clock. The wrapper is configured over JTAG, like everything else.

2. Problem and Project Idea

Problem Statement

Design, verify and demonstrate on the AUP ZU3 a memory BIST system built from SystemVerilog test IPs and controlled only through a JTAG test access port and an IEEE 1687 network. A programmable BIST controller must run March test algorithms on three different memories through local collars, in a slow and an at-speed clock mode. Faults injected on purpose must be detected and logged, and a fault-coverage experiment must measure how well each algorithm and each clock mode detects each type of fault.

Architecture (Figure 2.1). The **tester** (Vitis software on the PS) drives the JTAG pins TCK, TMS, TDI and TDO through GPIO. The student's **TAP controller** (IEEE 1149.1) decodes the JTAG protocol. Its `IJTAG` instruction connects the **IEEE 1687 network**: a chain of five Segment Insertion Bits (SIBs), one per instrument group (clock/mode, BIST controller, and **one SIB per memory**). Opening a SIB inserts that group's Test Data Registers (TDRs) into the scan path. The TDRs are the only interface to the *instruments*:

- the **BIST controller**, which executes March programs and broadcasts each March element to the **collar (memory wrapper)s**;
- the collars, which run the element on their memory, compare and log failures;
- the **fault-injection wrappers** between collars and memories;
- the **clock/mode IP**, which selects *functional*, *BIST-slow* or *BIST-at-speed* mode and switches the memory clocks without glitches.

In functional mode a **functional traffic generator** uses the memories as an ordinary design would, which shows that the test logic is transparent.

The experiment. The tester runs a *fault campaign*: for every algorithm, fault type and clock mode it injects many faults at random positions over JTAG, runs BIST and counts the detections. The result is a fault coverage table, compared with the theory of March tests [B1, A2], plus a slow-versus-at-speed comparison that shows why at-speed testing matters.

The main challenge is verification. Every IP must be verified on its own, and the whole system end to end *through the JTAG pins only*, against reference models of the scan network and of faulty memories (Chapter 9).

2.1 How the test access works: a short guide

This section explains, step by step, how a few JTAG pins can control the whole system. The student must understand it well before designing the test access (Chapter 7). Good introductions are [B2] (TAP) and [A1] (IEEE 1687).

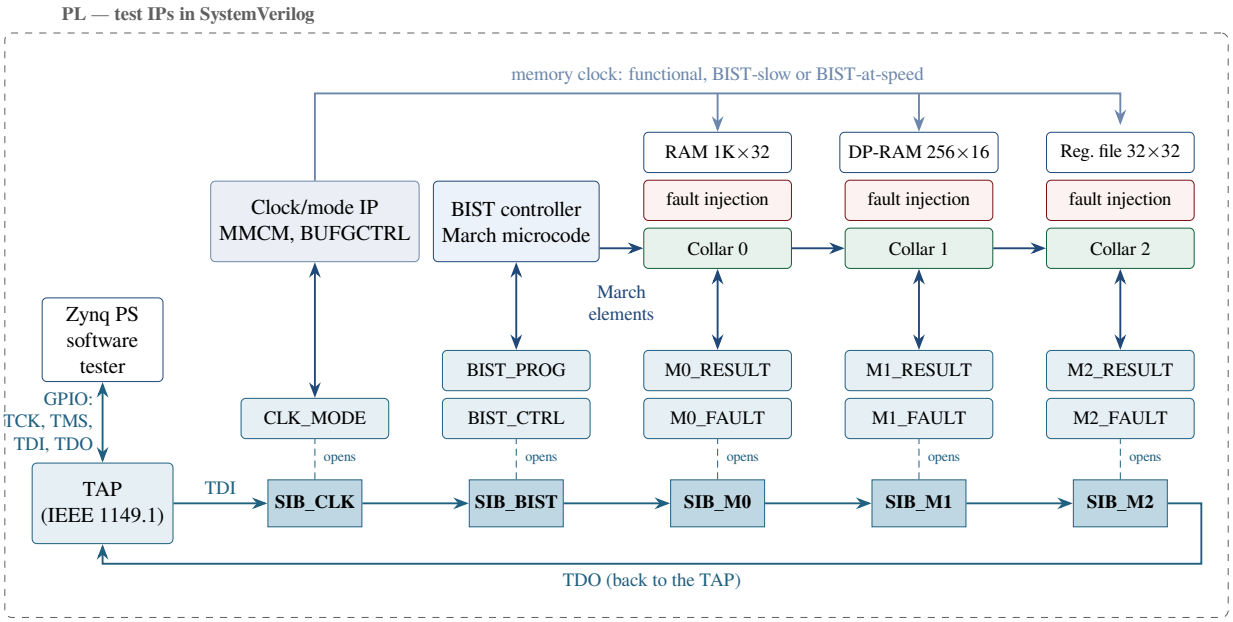


Figure 2.1: System architecture. Bottom: the scan chain of SIBs (one per instrument group, one per memory); middle: the TDRs each SIB opens; top: the instruments they control. Dashed lines: "opens"; solid: data between TDRs and instruments.

1. Four wires and a state machine

The tester uses four signals: **TCK** (test clock), **TMS** (mode select), **TDI** (data in) and **TDO** (data out). On every rising edge of TCK the TAP controller moves to its next state according to TMS (Figure 2.2). The right column handles the *instruction register* (IR), which selects what is connected between TDI and TDO; the middle column handles the selected *data register* (DR). In both columns the work is done in three states:

- **Capture**: load the current values from the hardware into the shift register;
- **Shift**: on every TCK, move the register by one bit: a new bit enters from TDI, the last bit leaves on TDO;
- **Update**: copy the shifted-in values to the outputs, where the hardware sees them.

Five TCK cycles with TMS = 1 always reach *Test-Logic-Reset*, from any state.

2. A test data register: capture, shift, update

Each TDR bit has a *shift bit* and an *update bit* (Figure 2.3). During shifting only the shift bits move, so the instrument does not see the intermediate values. At Update-DR, all update bits change at once. At Capture-DR, the shift bits load status values from the instrument (e.g. "done", a fail count), which are then shifted out on TDO. One DR scan therefore *writes* new control values and *reads* status at the same time.

3. A SIB opens a door in the scan path

A SIB is a one-bit TDR whose update bit controls a multiplexer (Figure 2.4). When the SIB is *closed* (0), scan data passes straight to the SIB bit, and the segment behind it is skipped. When it is *open* (1), the data first

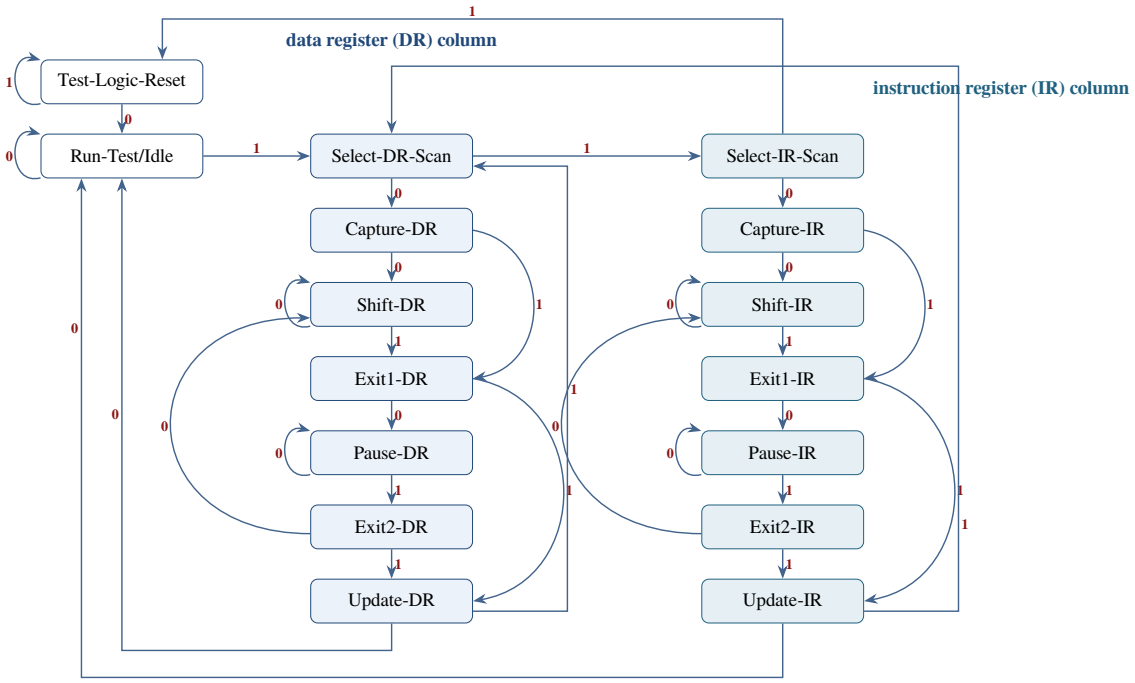


Figure 2.2: The 16-state TAP controller (IEEE 1149.1). Numbers on the arrows are the value of TMS.

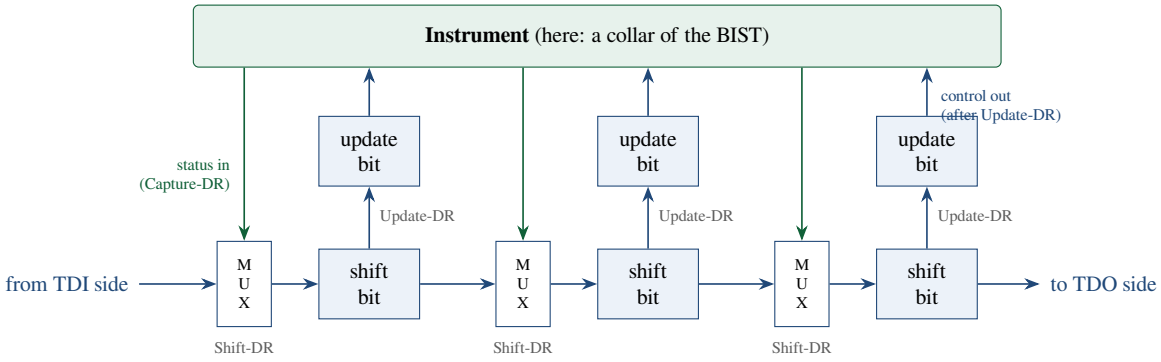


Figure 2.3: Three bits of a TDR. Shift bits form the scan path; update bits drive the instrument; capture loads status from it.

passes through the segment, here the TDRs of one memory. The tester opens a SIB by shifting a 1 into it; the change takes effect at Update-DR, so the *next* scan sees the longer path.

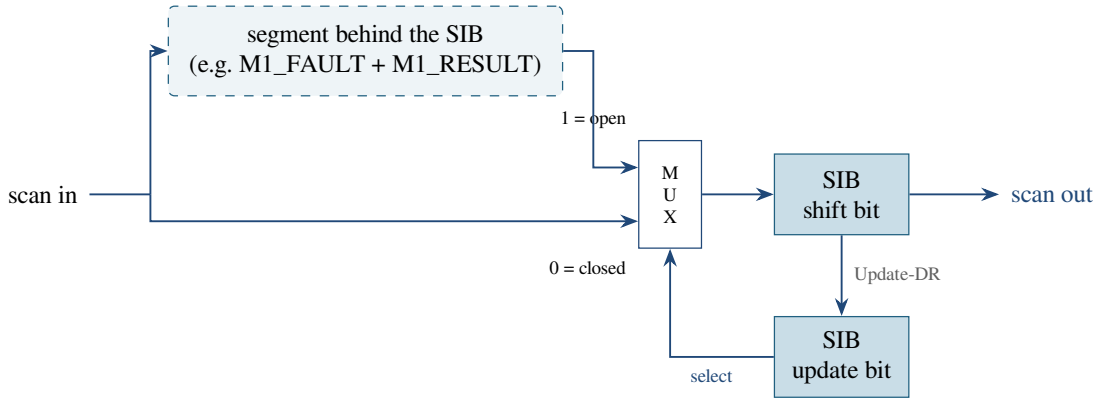


Figure 2.4: A SIB: its update bit selects whether the segment behind it is part of the scan path.

4. Example: reading the result of RAM 1

With one SIB per memory, the tester opens only what it needs (Figure 2.5). Table 2.1 lists the full sequence of JTAG operations. The network layer of the software tester (Chapter 8) computes these bit strings automatically.

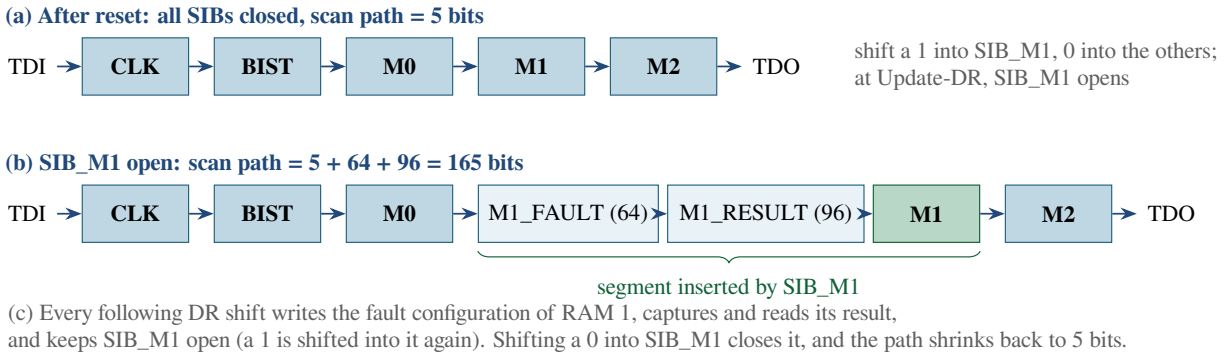


Figure 2.5: The active scan path before and after opening SIB_M1.

Table 2.1: JTAG operations to run BIST and read the result of RAM 1.

Step	JTAG operation	Effect
1	5 × TMS = 1	TAP reset; all SIBs closed
2	IR scan: `IJTAG`	the IEEE 1687 network is connected between TDI and TDO
3	DR scan, 5 bits	open SIB_BIST and SIB_M1
4	DR scan, 5 + 68 + 160 bits	write program/start into BIST_CTRL and BIST_PROG; write a fault into M1_FAULT
5	DR scans (repeated)	poll `done` in BIST_CTRL (captured at every scan)
6	DR scan	capture M1_RESULT: fail count, first fail-log entry; shift it out on TDO
7	DR scan	close all SIBs (shift 0s into them)

Key Point

Everything in this system, from loading a March program to reading a fail log, is done by IR and DR scans through these four pins. Understanding Capture–Shift–Update and SIBs is the key to the whole project.

3. Goals and Expected Outcomes

3.1 Project goals

1. **Specify the system:** memories under test, fault types, March algorithms, controller instruction format, TAP instructions, the IEEE 1687 network (SIBs and TDRs) and the clock modes.
2. **Design the test IPs** in SystemVerilog and package each as a Vivado IP: TAP controller, SIB and TDR cells, programmable BIST controller, collars, fault-injection wrappers, clock/mode controller, functional traffic generator.
3. **Support at least five algorithms** (MATS+, March X, March C−, March SS, checkerboard) as microcode programs, the **eleven fault types** of Section 5.2, and **three modes**: functional, BIST-slow, BIST-at-speed.
4. **Cross clock domains safely** between TCK, the BIST clock and the functional clock, and prove it in simulation and with Vivado's CDC report.
5. **Verify each IP** with its own self-checking testbench, and the complete system end to end through the JTAG pins with a class-based, constrained-random environment, reference models, assertions and functional coverage.
6. **Run the fault-coverage experiment** on the board over JTAG: at least 50 random faults per fault type, every algorithm, all memories, slow and at-speed. Compare the results with the theory.
7. **Implement the system** on the AUP ZU3 with the at-speed clock meeting timing (target 250 MHz **[confirm target frequency]**).

3.2 Learning outcomes

On completing the project the student will be able to:

- explain memory fault models and why March tests detect them;
- explain and implement the IEEE 1149.1 TAP and an IEEE 1687 network;
- design a programmable controller (instruction format, sequencer) in SystemVerilog;
- design clock-domain crossings and glitch-free clock switching on an FPGA;
- package designs as reusable IP and assemble them in IP Integrator;
- build a layered, class-based SystemVerilog testbench with reference models, SystemVerilog Assertions (SVA) and functional coverage;

- plan and analyse an experiment, and present the results as tables and plots.

4. Scope, MVP and Constraints

4.1 Minimum viable result (MVP)

- Three memories under test (Table 5.1), each with a collar and a fault-injection wrapper (four fault slots, eleven fault types).
- A **programmable** BIST controller (instruction memory, sequencer, data backgrounds, cycle counter); fail logs and fail counters in every collar.
- An IEEE 1149.1 **TAP controller** (instructions BYPASS, IDCODE, IJTAG) and an **IEEE 1687 network** of SIBs and TDRs (Table 7.2) giving access to all instruments.
- A **clock/mode IP**: functional, BIST-slow and BIST-at-speed modes; glitch-free clock switching; safe clock-domain crossings.
- A functional traffic generator that exercises the memories in functional mode.
- Every block packaged as a Vivado IP and the system assembled in IP Integrator.
- A **software tester** on the PS: bit-banged JTAG over GPIO, a network-access layer, BIST commands, the fault campaign; results printed over UART.
- Unit testbenches for every IP; an end-to-end environment driven through the JTAG pins; reference models, SVA, functional coverage; a regression script.
- The fault-coverage table (slow and at-speed) and its comparison with the theory.

4.2 Optional (if time allows)

- **External JTAG:** bring the TAP pins to a Pmod connector and drive them from a PC with a USB-JTAG adapter and OpenOCD, instead of the PS.
- **ICL and PDL:** describe the network in ICL and write the test procedures in PDL [U2]; generate the tester's commands from them.
- A **hard-wired** BIST controller and a comparison with the programmable one.

4.3 Out of scope (not required)

- IEEE 1500 core wrappers; boundary scan of the FPGA pins; use of the FPGA's own JTAG port (BSCANE2); AXI access to the instruments.
- Built-in self-repair, ROM/MISR testing, data-retention faults, port-interaction faults. See Chapter 17.

4.4 Constraints and assumptions

- One AUP ZU3 board is available to the student **[confirm board availability and lab access]**.
- The AUP ZU3 device (XCZU3EG) is supported by the free Vivado/Vitis edition (Vivado Simulator, IP Packager and IP Integrator included).
- The software tester toggles TCK through GPIO, so JTAG runs at well below 1 MHz. This is fine: shifting is slow, but the BIST itself runs on its own fast clock.
- Clocks are switched only with the FPGA's clock-buffer primitives (BUFGCTRL / BUFGMUX) [U3], never with logic gates, and only while BIST is idle.
- Memories are inferred from SystemVerilog (no memory IP cores).
- Instruction format, TAP instruction codes, network layout and TDR formats in Chapters 6–7 are a proposal. The student may refine them with the supervisor's agreement and must document the final version.

5. Memories, Fault Models and Test Algorithms

5.1 Memories under test

The three memories differ in type, size and width. One controller must therefore serve memories with different geometry, and the coverage results can be compared across memory types.

Table 5.1: Memories under test (proposal).

Memory	Size	FPGA resource	Notes
MUT0 single-port RAM	1024×32	block RAM [U4]	largest memory, dominates the test time
MUT1 dual-port RAM	256×16	block RAM [U4]	tested through port A (port B optional)
MUT2 register file	32×32	LUTRAM [U5]	asynchronous read: a different read timing

5.2 Fault models

A *fault model* describes how a defective memory behaves at its ports, independent of the physical cause [B1, B3]. The project supports the following eleven fault types. Each one is applied to one bit (the *victim*) of one word; coupling faults also have an *aggressor / victim* bit.

The SPD fault is detected only by algorithms that read a cell directly after writing it (such as the $w0, r0$ pairs of March SS), and only at speed. This makes it the clearest demonstration of why at-speed testing and the choice of algorithm both matter.

Coupling faults can be *inter-word* (aggressor and victim in different words) or *intra-word* (two bits of the same word). Intra-word faults are only detected if the test uses several data backgrounds (Section 5.4). This effect is one of the things the experiment should show.

Table 5.2: Fault types to inject.

Class	Type	Behaviour
SAF	SAF0, SAF1	the victim bit always reads 0 (or 1), whatever is written
TF	TF↑, TF↓	the victim cannot change from 0 to 1 (or from 1 to 0)
CF	CFin	a given transition (↑ or ↓) of the aggressor *inverts* the victim
	CFid	a given transition of the aggressor *forces* the victim to 0 or 1
	CFst	while the aggressor is in a given state (0 or 1), the victim is forced to 0 or 1
AF	AF-none	address A reaches no cell: writes are lost, reads return 0
	AF-alias	address A reaches the cell of address B instead of its own
	AF-multi	address A reaches its own cell *and* the cell of B (reads return the AND of both)
Speed	SPD	write-recovery fault: in BIST-at-speed mode, a read of the victim cell *immediately* after a write to it returns the *old* value of the victim bit (the cell is too slow); at slow speed it works

5.3 March tests

A March test is a list of *March elements*. Each element visits all addresses in ascending ($\Uparrow$), descending ($\Downarrow$) or any ($\Updownarrow$) order, and applies the same operations to each address before moving on: `w0`/`w1` write the background or its inverse, and `r0`/`r1` read and compare with it [A2, A3]. The test length is counted in operations per address, so for n addresses March C$-$ needs $10n$ operations.

Table 5.3: Algorithms to implement and the faults they detect in theory (bit-oriented memory, single faults).

Algorithm	March elements	Length	Detects
MATS+	$\Updownarrow\,(w0);\Uparrow\,(r0,w1);\Downarrow\,(r1,w0)$	$5n$	SAF, AF; TF only partly
March X	$\Updownarrow\,(w0);\Uparrow\,(r0,w1);\Downarrow\,(r1,w0);\Updownarrow\,(r0)$	$6n$	SAF, AF, TF, CFin
March C$-$	$\Updownarrow\,(w0);\Uparrow\,(r0,w1);\Uparrow\,(r1,w0);\Downarrow\,(r0,w1);\Downarrow\,(r1,w0);$ $\Updownarrow\,(r0)$	$10n$	SAF, AF, TF, CFin, CFid, CFst
March SS	$\Updownarrow\,(w0);\Uparrow\,(r0,r0,w0,r0,w1);\Uparrow\,(r1,r1,w1,r1,w0);$ $\Downarrow\,(r0,r0,w0,r0,w1);\Downarrow\,(r1,r1,w1,r1,w0);\Updownarrow\,(r0)$	$22n$	all simple static faults [A4]
Checkerboard	write checkerboard; read; write inverse; read	$4n$	SAF; not AF, CF only partly

5.4 Data backgrounds

The memories are *word-oriented*: one address holds 16 or 32 bits. "0" in a March test then means the data background pattern D, and "1" means its inverse $\overline{D}$. With the solid background (D = all zeros), neighbouring bits of a word always hold the same value, so coupling between them stays invisible. The standard remedy is to repeat the test with $\lceil \log_2 B \rceil + 1$ backgrounds for B-bit words: solid, then alternating patterns `0101`…, `0011`…, `00001111`…, … [B1]. The controller must support a background register and, ideally, an automatic loop over the backgrounds.

5.5 Fault-injection wrapper

Each memory is connected to its collar through a wrapper that can hold up to **four active faults** (four "fault slots"). Each slot is configured over JTAG through the memory's Mk_FAULT TDR with: type, victim address and bit, aggressor address and bit (coupling faults), transition direction or forced value, and an enable bit. In functional mode the wrapper applies the same faults, so a faulty memory stays faulty. This is useful for the demonstration. The SPD fault needs to know the current clock mode, which the wrapper receives as a static signal from the clock/mode IP.

Implementation hints. Several faults need the *old* content of a cell (TF, CF) or change a *second* cell (CF, AF-multi). Two approaches are possible, and the student chooses one in T1.3 and justifies it:

1. **Extra memory cycles:** the wrapper performs read-modify-write cycles and stalls the collar through a `ready` signal while it updates the victim;
2. **Victim overlay:** the wrapper keeps the victim bit's "faulty" value in a register and substitutes it on reads of the victim address.

Either way, the wrapper must be verified against a software model of a faulty memory (Chapter 9).

6. BIST Architecture

6.1 Division of work: controller and collars

The system is *hierarchical* [A5, B4]. The **controller** decides *what* to do: it fetches the March program, sends one March element at a time to all collars, and waits until every enabled collar has finished it. The **collars** decide *how* to do it for their memory: each has its own address generator (sized to its memory) and data generator (sized to its width), executes the element over all its addresses, compares read data and logs failures. Because the memories have different depths, the collars finish an element at different times. The controller starts the next element only when all of them report `elem_done`.

6.2 Instruction format (proposal)

Each March element is one 32-bit instruction in a small instruction memory (16 entries), written over JTAG through the program TDR.

Table 6.1: Proposed March instruction format.

Bits	Field	Meaning
[31]	`LAST`	last element of the program
[30:29]	`ORDER`	00 = ⇑, 01 = ⇓, 10 = ⇕ (executed as ⇑)
[28:26]	`NOPS`	number of operations in the element, 1–6
[23:12]	`OPS`	six 2-bit operations {R/W, D/$\overline{\text{D}}$}: `w0, w1, r0, r1`
[11:8]	`WAIT`	optional pause before the element (2^k cycles; for retention faults)
others	–	reserved (0)

Example: March C− as a program.

#	ORDER	OPS	LAST
0	⇕	w0	0
1	⇑	r0, w1	0
2	⇑	r1, w0	0
3	⇓	r0, w1	0
4	⇓	r1, w0	0
5	⇕	r0	1

The checkerboard test needs a background that alternates between addresses as well as between bits. The student either adds an instruction bit for “invert data on odd addresses” or explains how to express it with the available fields.

6.3 Collar

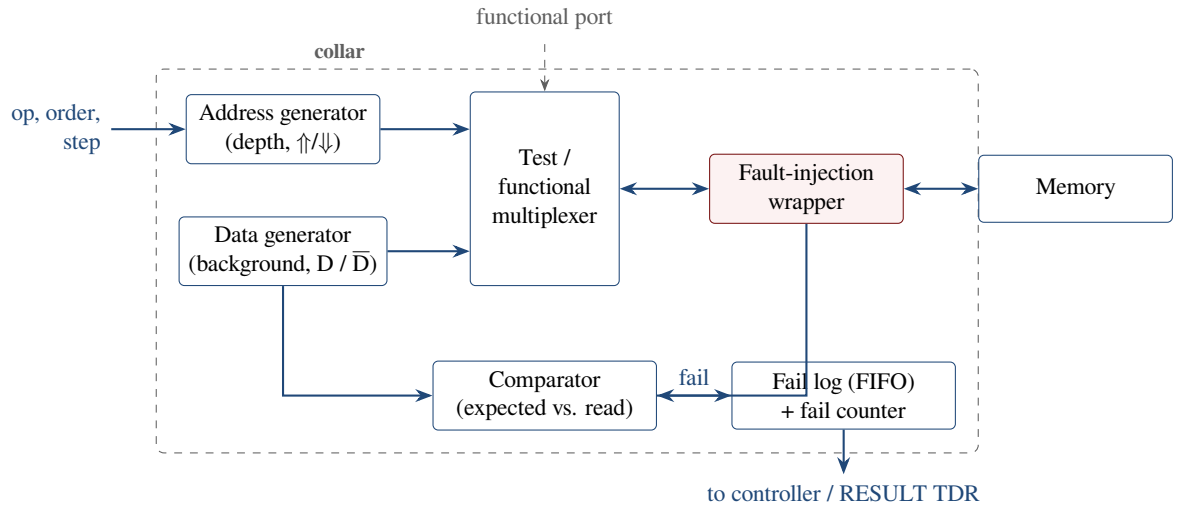


Figure 6.1: Inside a collar.

Each collar (Figure 6.1) contains:

- a **test/functional multiplexer**: in functional mode the memory belongs to the functional port; during BIST the collar owns it, and functional accesses are blocked and flagged;
- an **address generator** (up/down counter up to the memory depth) and a **data generator** (background D or $\overline{D}$, width-adapted);
- a **comparator** that checks every read against the expected value, taking the memory’s read latency into account (block RAM: one cycle; LUTRAM: asynchronous);
- a **fail log**: a FIFO of 16 entries plus a fail counter and an overflow flag.

Table 6.2: Proposed fail-log entry.

Field	Content
element, operation	position in the March program (e.g. element 3, operation 1)
address	failing address
expected, actual	expected and read data (full word)
background	index of the background in use

6.4 Optional: hard-wired controller

A second controller with the same collar interface, where MATS+, March C− and checkerboard are fixed in an FSM and selected by a register. The student compares the two controllers (LUTs, flip-flops, maximum frequency, test time, flexibility), which is a small design-space study for the diploma.

6.5 Packaging as IP blocks

Each block is packaged with the Vivado IP Packager [U6] as a reusable IP with documented parameters (e.g. memory depth and width, number of fault slots, TDR lengths) and assembled in an IP Integrator block design [U7]:

IP	Main parameters
`jtag_tap`	IR length, IDCODE value
`ijtag_sib`, `ijtag_tdr`	TDR length, reset value, read-only/read-write fields
`mbist_ctrl`	instruction-memory depth, number of collars
`mbist_collar`	memory depth, width, read latency, fail-log depth
`fault_wrapper`	memory depth, width, number of fault slots
`clk_mode_ctrl`	clock frequencies (via Clocking Wizard [U8])
`func_traffic_gen`	memory depth, width, pattern

7. Test Access and Clock Modes

7.1 TAP controller (IEEE 1149.1)

The TAP controller is the standard 16-state FSM driven by TMS on the rising edge of TCK [U1, B2]. It shifts an instruction into the instruction register (IR) and then shifts data through the data register selected by that instruction. Test-Logic-Reset (five TMS = 1 clocks) must always bring the TAP and the network to a known

Table 7.1: Proposed TAP instructions (4-bit IR).

Code	Instruction	Selected data register
`1111`	BYPASS	1-bit bypass register (mandatory in 1149.1)
`0001`	IDCODE	32-bit identification register (version, part, manufacturer fields)
`0100`	IJTAG	the IEEE 1687 network (its current active scan path)

state: all SIBs closed, all instruments idle.

7.2 IEEE 1687 network

A SIB is a one-bit register in the scan path. When it is *closed*, the segment behind it is skipped. When it is *opened* (a 1 is shifted in and *updated*), its segment, here one TDR, is inserted into the active scan path for the next shifts [U2, A1]. The tester opens only the SIBs it needs, so the scan path stays short. TDR bits are shifted in the TCK domain and take effect at *Update-DR*. Status bits are *captured* into the TDR at *Capture-DR*.

Table 7.2: Proposed IEEE 1687 network: one SIB per instrument group, one SIB per memory.

SIB	TDRs behind it (bits)	Fields
SIB_CLK	CLK_MODE (8)	write: mode (functional / BIST-slow / BIST-at-speed); capture: current mode, clock locked, switch done
SIB_BIST	BIST_CTRL (32)	write: start, abort, memory enable mask, background, background loop; capture: busy, done, pass/fail per memory, cycle count
	BIST_PROG (36)	instruction-memory address (4) + March instruction (32), written at Update-DR
SIB_Mk (k = 0, 1, 2)	Mk_FAULT (64)	fault slot, type, victim address/bit, aggressor address/bit, value/direction, enable
	Mk_RESULT (96)	capture: fail count, overflow, one fail-log entry (element, operation, address, expected, actual); Update-DR pops the entry

With all SIBs closed the scan path is 5 bits long. Opening one memory's SIB adds 160 bits, so the tester never shifts through the registers of memories it does not need. Adding a fourth memory means adding one more SIB with the same two TDRs. The structure scales, which is the main idea of IEEE 1687.

7.3 Clock domains and modes

The system has three clock domains:

- **TCK**: JTAG shifting, TAP, SIBs and the shift stages of the TDRs (slow, driven by the tester);
- **BIST clock**: controller, collars, fault wrappers and memories during test. It is either a slow clock (e.g. 25 MHz) or the fast at-speed clock (target 250 MHz), generated by an MMCM (Clocking Wizard [U8]);
- **functional clock**: the memories and the traffic generator in functional mode (e.g. 100 MHz).

Table 7.3: Modes.

Mode	Memory clock	Who uses the memories
Functional	functional clock	traffic generator; collars transparent
BIST-slow	slow clock	collars under control of the BIST controller
BIST-at-speed	fast clock	same, at the target frequency; SPD faults become visible

Rules for the design [U3, A6]:

- The memory clock is switched with a **glitch-free clock buffer** (BUFGCTRL / BUFGMUX), never with logic. Switching is allowed only when BIST is idle and the traffic generator is paused. The clock/mode IP performs the sequence pause → switch → confirm, and reports "switch done" in CLK_MODE.

- Every TDR has a **shift stage** (TCK) and an **update stage**. Values cross from TCK into the BIST domain through a **handshake synchronizer** (request/acknowledge), not through single flip-flops. Multi-bit values are transferred only as a whole, on a request.
- Status and results go back into the TCK domain through synchronizers and are stable when captured, because the tester reads them only when `done` is set.
- Resets are synchronized to each domain. Vivado's CDC report [U9] must show no unsafe crossings.

8. Software Tester and the Fault-Coverage Experiment

8.1 Hardware platform

The test IPs are assembled in an IP Integrator block design [U7] with the Zynq UltraScale+ PS [U10]. The only connection between the PS and the test system is a **GPIO** (PS EMIO GPIO or an AXI GPIO IP [U11]) wired to the TAP pins TCK, TMS, TDI (outputs) and TDO (input). The PS has no other access to the instruments. Constraints, implementation, timing closure at the at-speed frequency [U9] and the CDC report complete the platform.

8.2 Software tester (Vitis, MVP)

A bare-metal C application [U12] acts as the tester. It is organised in layers, like professional test software:

1. **JTAG layer**: `tap_reset()`, `shift_ir()`, `shift_dr()` by toggling the GPIO pins; reading the IDCODE as the first sanity check.
2. **Network layer**: keeps a model of the IEEE 1687 network (which SIBs are open, what the active scan path is), and provides `write_tdr(name, value)` and `read_tdr(name)` by computing the right SIB settings and bit positions. This is the job of PDL commands such as `iWrite`/`iRead`/`iApply` in IEEE 1687.
3. **Test layer**: algorithm library (the March programs of Table 5.3), fault configuration, mode selection, run BIST, read pass/fail, fail logs and cycle count.
4. **Campaign runner** for the experiment below, and a UART menu for the live demonstration.

8.3 The fault-coverage experiment

For every algorithm × fault type × memory × mode (BIST-slow, BIST-at-speed), the campaign runner:

1. injects one fault at a random position (random victim, and random aggressor for coupling and address faults, both inter-word and intra-word);
2. runs the algorithm (with the solid background, then with all backgrounds);

3. records whether the fault was detected, and whether the logged address points to the victim or the aggressor (diagnosis);
4. repeats this for at least 50 random positions.

The output is a CSV table of detection rates (Table 8.1), plus the test time of each algorithm in cycles and in microseconds for both clocks. The student explains every difference from the theory of Table 5.3. For example: why does MATS+ detect some, but not all, transition faults? Why do only March SS and only at-speed runs detect SPD faults? How much do the extra backgrounds help with intra-word coupling?

Table 8.1: Shape of the expected result (values in % detected; to be measured, separately for slow and at-speed mode).

	SAF0	SAF1	TF↑	TF↓	CFin	CFid	CFst	AF-n	AF-a	AF-m	SPD
MATS+											
March X											
March C−											
March SS											
Checkerboard											

9. Verification Plan

A BIST system that has a bug is worse than none: it may report “pass” for a faulty memory. Verification is therefore the main part of this project. It follows the usual industrial approach: **plan first, then verify bottom-up**. Every testbench is **self-checking** [B5].

9.1 Key idea: reference models

The heart of the verification is a set of SystemVerilog **reference models**:

- a *faulty-memory model*: an array plus a list of active faults, which implements the behaviour of Table 5.2 exactly as the hardware wrapper should;
- a *March interpreter*: executes the same program as the controller on the model, with the same address orders and backgrounds, and produces the list of failing reads;
- a *network model*: knows which SIBs are open, the active scan path, and where every TDR field sits in it. The testbench uses it to drive the DUT through the JTAG pins only, exactly as the software tester does.

For any program and set of faults, the model predicts the exact fail logs, fail counts, pass/fail and number of cycles. The scoreboard compares them with the hardware. The same model is useful in the fault-coverage experiment, to explain the measured results.

9.2 Verification levels

Table 9.1: Verification levels, methods and pass criteria.

Level / DUT	**Stimulus and checking**	**Done when**
Fault-injection wrapper	Random read/write traffic with every fault type at random positions; compare every read with the faulty-memory model.	0 mismatches; every fault type, transition direction and value covered.
Collar	Element commands with all orders and operations on a fault-free and a faulty memory; check addresses, data, comparator decisions and fail-log entries.	All orders, operations, memory types and fail-log full/overflow covered.
Controller	Random valid programs (1–16 elements); collars replaced by simple models with random completion times; check element sequence and `LAST` handling.	Program space covered (orders, op counts, waits); 0 errors.
TAP controller	JTAG BFM with random TMS sequences; check every state transition against a model of the 16-state FSM; IR and IDCODE reads; BYPASS length.	All 16 states and all transitions covered; 0 errors.
IEEE 1687 network	Random sequences of SIB openings/closings and TDR writes/reads; compare scan-path length and contents with the network model.	All SIB combinations and TDRs covered.
Clock/mode IP and CDC	Mode changes at random times, with BIST idle and (illegally) busy; random TCK/BIST-clock ratios; check handshakes and that illegal switches are refused.	No lost or duplicated transfers; Vivado CDC report clean.
Top level	Class-based environment (Figure 9.1) driving only the JTAG pins: library and random March programs, random fault sets, all backgrounds, all modes, functional traffic before and after BIST.	Coverage goals of Section 9.4; 0 scoreboard errors over the regression.
On the board	Integrated Logic Analyzer (ILA) [U13] on one collar; results of selected runs compared with the simulated reference model.	Board and simulation give identical fail logs.

9.3 Top-level verification environment

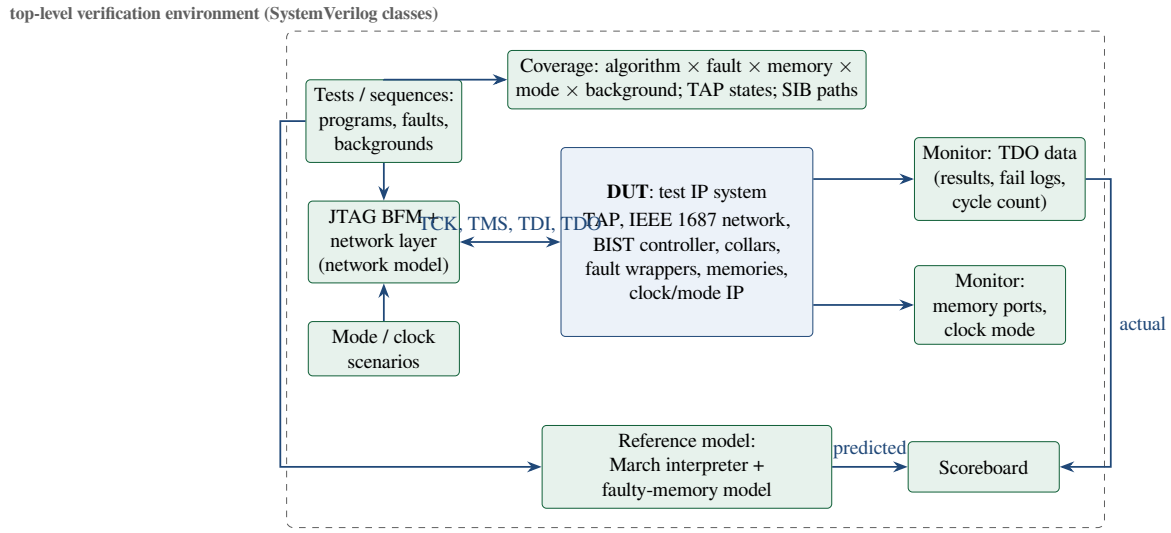


Figure 9.1: Structure of the top-level verification environment.

- **JTAG driver**: a bus functional model that drives TCK, TMS and TDI and samples TDO, with tasks for TAP reset, IR and DR shifts; on top of it the network layer (TDR read/write via the network model).
- **Sequences**: select modes, load programs (library or constrained-random valid programs), configure 0–4 faults per memory with constrained-random parameters, select backgrounds, run BIST, read results; functional traffic before and after.
- **Monitors and scoreboard**: compare every fail-log entry, counter, pass/fail flag and the cycle count with the reference model.
- **Assertions (SVA)** [B6]: TAP state follows TMS; TDR update stages change only at Update-DR; SIB state changes only at Update-DR; clock switching only while BIST is idle; CDC handshakes complete (request held until acknowledge); no functional access reaches a memory during BIST; addresses stay within each memory's depth; `done` only after the `LAST` element; a fail-log entry is written exactly when the comparator detects a mismatch.
- **Functional coverage**: covergroups and crosses for the items in Section 9.4.

A Universal Verification Methodology (UVM) version [B7] is optional; the required version uses plain SystemVerilog classes.

9.4 Coverage goals

- Every algorithm × every fault type × every memory × both BIST modes.
- Fault position: first address, last address, middle; LSB, MSB; aggressor below and above the victim (this decides whether ⇑ or ⇓ elements detect a coupling fault); inter-word and intra-word coupling.
- Backgrounds: every background pattern, and the background loop.

- Several simultaneous faults; no fault; fail-log overflow.
- Controller: all orders, operation counts 1–6, waits, programs of length 1 and 16.
- Test access: all TAP states and transitions; all instructions; every SIB open/closed combination; every TDR written and read.
- Modes: every mode, every mode-to-mode transition, and SPD faults in both BIST modes.
- Target: **100%** of the plan, or every uncovered item explained in the report.

9.5 Regression and reporting

All tests run from one script (the Vivado Simulator in batch mode, started from Tcl or a Makefile) with several random seeds [U14]. The script prints a PASS/FAIL summary and produces a coverage report. The regression must pass before every milestone review.

10. Tasks

Individual topic: *Design and verification of a JTAG-accessible, programmable memory BIST system with IEEE 1687 access, at-speed test modes and fault injection on the AMD AUP ZU3.*

10.1 Specification

T1.1 Fix the memories, fault types and algorithms; write the expected-detection table from the literature (Table 5.3), including SPD faults in both modes.

T1.2 Write the verification plan: features, test list, assertions, coverage items (refine Chapter 9).

T1.3 Fix the March instruction format, the collar interface, the fail-log format and the fault-wrapper approach.

T1.4 Fix the TAP instructions, the IEEE 1687 network (SIBs, TDR layouts), the clock frequencies and the mode-switch sequence.

10.2 Design (SystemVerilog RTL, packaged as IPs)

T2.1 Memories (inferred) and the fault-injection wrapper with four slots and eleven fault types.

T2.2 Collar: test/functional multiplexer, address and data generators, comparator, fail log.

T2.3 Programmable BIST controller: instruction memory, sequencer, backgrounds, cycle counter.

T2.4 TAP controller (BYPASS, IDCODE, IJTAG).

T2.5 IEEE 1687 network: SIB and TDR cells; the network of five SIBs (clock, BIST, one per memory) and its TDRs; CDC handshakes to the instruments.

T2.6 Clock/mode IP (Clocking Wizard, BUFGCTRL switching, pause/switch/confirm sequence) and the functional traffic generator.

T2.7 Package all blocks as Vivado IPs; build the IP Integrator block design.

T2.8 (optional) Hard-wired BIST controller and comparison with the programmable one.

10.3 Verification

T3.1 Reference models: faulty-memory model, March interpreter, network model.

T3.2 Unit testbenches with SVA: fault wrapper, collar, controller, TAP, network, clock/mode IP.

T3.3 End-to-end environment through the JTAG pins: JTAG BFM, network layer, sequences, monitors, scoreboard, SVA, coverage.

T3.4 Regression script with coverage report; coverage closure; Vivado CDC report clean.

10.4 Implementation, tester and experiment

T4.1 Constrain, implement and close timing at the at-speed frequency; report resource usage.

T4.2 Software tester: JTAG layer, network layer, test layer, UART menu.

T4.3 Fault-coverage campaign (slow and at-speed); tables and plots; comparison with theory and simulation.

T4.4 (optional) External JTAG through Pmod pins with a USB-JTAG adapter and OpenOCD; ICL/PDL description.

Key Point

The diploma is mainly judged on the **system architecture and RTL** (BIST, test access, clock modes), the **verification** (reference models, end-to-end environment through JTAG, assertions, coverage, CDC) and the **fault-coverage experiment** with its analysis.

11. Learning Path

11.1 Prerequisites

The student is expected to already have:

- Digital logic: combinational and sequential circuits, Finite-State Machines (FSMs), timing diagrams. *Self-check:* design a counter and a small FSM and draw their waveforms. *If weak:* [B8], ch. 2–3.
- An HDL (Verilog or SystemVerilog): modules, always blocks, simulation. *Self-check:* write and simulate a parameterised register. *If weak:* [B8], ch. 4.

- C programming (arrays, structs, bit operations), for the Vitis application.
- Basic probability, for planning the experiment (random fault positions, sample size).

11.2 What to learn

Aware — knows what it is and when it is used; Understand — can explain it and read existing work; Apply — can use it independently to build.

Table 11.1: Knowledge map.

Topic	Level	Where it is used in the project	Resources
Memory testing			
Memory organisation and why memories fail	Understand	context, fault models	[B1]; [B9], memory test chapter
Functional fault models (SAF, TF, CF, AF)	Apply	fault wrapper, reference model	[B1]; [B3]
March notation and algorithms	Apply	programs, controller	[A2]; [A3]
Word-oriented memories and data backgrounds	Apply	data generator, experiment	[B1]
Fault coverage and its analysis	Apply	experiment	[B1]; [A4]
BIST architecture			
Memory BIST architectures: controller, collars, distributed control	Understand	system architecture	[B4], memory test chapter; [A5]
Built-in self-repair, MISR, IEEE 1500	Aware	extensions	[B4]; [U15]
Test access			
IEEE 1149.1 TAP: state machine, IR/DR, BYPASS, IDCODE	Apply	TAP controller, JTAG layer	[B2]; [U1]; [B4], boundary-scan chapter
IEEE 1687: SIBs, TDRs, scan-path reconfiguration	Apply	network, network model	[A1]; [U2]
ICL and PDL	Aware	optional ICL/PDL description	[U2]
Clocks			
FPGA clocking: MMCM, global buffers, glitch-free switching	Apply	clock/mode IP	[U3]; [U8]

Topic	Level	Where it is used in the project	Resources
Clock-domain crossing: synchronizers, handshakes, reset sync	Apply	TDR update/capture, mode switch	[A6]
At-speed testing and timing-related faults	Understand	BIST-at-speed mode, SPD faults	[B4]; [B9]
SystemVerilog design			
Synthesizable SystemVerilog: packages, interfaces, structs, enums	Apply	all RTL	[B10]
Small programmable controllers (fetch, decode, sequence)	Apply	BIST controller	[B8], ch. 7 (concepts)
Inferred block RAM and LUTRAM	Apply	memories under test	[U4]; [U5]
Packaging IP and IP Integrator	Apply	IP system	[U6]; [U7]
Verification			
Classes, randomization, constraints	Apply	testbenches, random programs and faults	[B5], ch. 5–6
Layered testbench, scoreboard, reference model	Apply	top-level environment	[B5], ch. 8, 11
SystemVerilog Assertions	Apply	protocol and design checks	[B6]
Functional coverage	Apply	coverage plan	[B5], ch. 9
UVM concepts	Aware	optional UVM environment	[B7]
Zynq platform and tools			
Zynq UltraScale+ PS/PL, EMIO / AXI GPIO	Understand	tester connection	[U10]; [U11]
Vivado Simulator, batch mode and coverage	Apply	regression	[U14]
Timing analysis and closure; CDC report	Apply	at-speed implementation	[U9]
Vitis bare-metal application, GPIO bit-banging	Apply	software tester, campaign	[U12]

11.3 Tools and skills

- **Vivado** (Vivado Simulator, synthesis, implementation, ILA) and **Vitis**: free edition, same version on all machines **[fix the tool version]**.

- **git**: all code, scripts and documents in a repository from the first week.
- **Scripting** (Tcl or Python): regression, build, and processing of the campaign CSV into tables and plots.
- The **IEEE 1800** SystemVerilog standard [U16] as reference when a book is not enough.

12. Getting Started

12.1 First meeting checklist

- Repository created; supervisor has access; folder structure agreed (`rtl/`, `tb/`, `sw/`, `scripts/`, `doc/`).
- AUP ZU3 board, power supply and cables handed over; lab access **[lab / room]**.
- Weekly meeting slot and communication channel.

12.2 Environment setup

1. Install Vivado and Vitis (agreed version) and the AUP ZU3 board files.
2. Build a minimal Zynq block design, generate the bitstream and run a "hello world" Vitis application that prints over UART.
3. Run a small SystemVerilog simulation from a script in batch mode (no GUI).

12.3 Warm-up exercises

Five short exercises, from simple to project-like. Each one practises a skill needed later, and each is kept in the repository with its own testbench.

Warm-up exercise 1: Finite-state machine: the TAP controller — Est. time: 6 h

Implement the 16-state IEEE 1149.1 TAP state machine [B2] with `typedef enum` and `always_ff`/`always_comb`. Write a testbench that applies random TMS sequences and compares the state with a simple table-based model, and check that five TMS = 1 clocks always reach Test-Logic-Reset.

Done when: all 16 states and 32 transitions are reached in simulation with no mismatch; the student can draw the state diagram from memory.

Warm-up exercise 2: Memories: RAM and FIFO — Est. time: 8 h

(a) Write a parameterised single-port and a simple dual-port RAM so that Vivado infers block RAM (check the synthesis report). (b) Build a synchronous FIFO on top of the dual-port RAM. Write a class-based testbench with random push/pop traffic, a queue-based reference model, two SVA properties (no write when full, no read when empty) and one covergroup (full, empty, simultaneous push and pop).

Done when: the synthesis report shows BRAM, not LUTs; the FIFO testbench reports PASS for 3 seeds and the coverage shows full and empty were reached.

Warm-up exercise 3: Hard-coded MATS+ on one RAM Est. time: 8 h

Write an FSM that runs MATS+ ($\Updownarrow\ (w0); \Uparrow\ (r0, w1); \Downarrow\ (r1, w0)$) on a 64×8 inferred RAM and reports pass/fail and the first failing address. Check in simulation that it passes on a good RAM and that it needs exactly $5n$ operations.

Done when: pass on the fault-free RAM; the cycle count matches 5×64 operations plus the read latency.

Warm-up exercise 4: First fault injection Est. time: 6 h

Put a small wrapper between the MATS+ FSM and the RAM that can force one chosen bit of one chosen address to 0 or 1 (stuck-at). Write a testbench that places the fault at random positions and checks that the FSM reports the right address.

Done when: 100 random stuck-at faults are all detected and located correctly; the student can explain which March element finds SAF0 and which finds SAF1.

Warm-up exercise 5: Bit-banged JTAG from the PS Est. time: 10 h

Add an instruction register, BYPASS and a 32-bit IDCODE to the TAP of Warm-up 1, connect its pins to a GPIO of the PS in a block design, and write a Vitis function that toggles TCK/TMS/TDI and reads TDO to shift the IR and read the IDCODE.

Done when: the IDCODE printed over UART equals the value in the RTL, and BYPASS adds exactly one bit to a shifted pattern.

12.4 Read first

1. [A2]: a short, readable introduction to March tests.
2. [B1], chapters on fault models and functional testing.
3. [B2], chapters on the TAP controller and instruction register.
4. [B5], chapters on classes, randomization and the layered testbench.

13. Work Plan and Milestones

The project runs over **30 weeks**. Weeks 14–15 fall on the winter holidays and the exam session. They are kept free as a buffer for catching up. The weekly checklist in Chapter 19 breaks this plan into tasks per week.

Table 13.1: Work plan (30 weeks).

Weeks	Phase / milestone	Checkpoint result
1–4	Onboarding: tools, reading, five warm-up exercises (Chapter 12)	Warm-ups accepted
5–6	Specification: memories, faults, algorithms, instruction format, TAP and IEEE 1687 network, clocks and modes; verification plan	Specification and verification plan approved
7–9	Memories + fault-injection wrapper (incl. SPD); faulty-memory model; wrapper testbench	Wrapper matches the model; code review 1
10–13	Collar and programmable controller; March interpreter; unit testbenches + SVA	BIST core verified; code review 2
14–15	Buffer (holidays, exams): catch up; read on IEEE 1687 and CDC	Nothing behind schedule
16–18	TAP, IEEE 1687 network, TDRs with CDC handshakes, clock/mode IP; network model; unit testbenches	Test access verified; code review 3
19–22	IP packaging, top-level integration; end-to-end environment through JTAG; regression; coverage closure; CDC report	Coverage goals met; RTL and testbench frozen
23–25	Implementation and at-speed timing; software tester; fault-coverage campaign (slow and at-speed)	Coverage tables measured; demo on the board
26–28	Writing the diploma document (including analysis of the experiment)	Full draft to the supervisor (week 28)
29–30	Revisions, slides, demo rehearsal	Final document and presentation ready

Note

Weekly meeting with the supervisor **[day / time]**. Before each meeting the student sends a short note: what was done, what is blocked, next steps, and the latest regression result. The tracker in Chapter 19 is updated at every meeting.

14. Deliverables

Table 14.1: Project deliverables.

Deliverable	Form	Due	Responsible
Warm-up exercises (code + testbenches)	git repository	week 4	Student
Specification: memories, faults, algorithms, instruction format, TAP instructions, IEEE 1687 network, clock modes	Markdown/PDF in the repository	week 6	Student
Verification plan	document in the repository	week 6	Student
SystemVerilog RTL packaged as Vivado IPs; block design	git repository, README	week 22	Student
Reference model, testbenches, assertions, coverage model, regression script	git repository; regression and coverage reports	week 22	Student
Vivado build script; software tester (JTAG, network and test layers, campaign runner)	git repository with build instructions	week 25	Student
Fault-coverage results	CSV + tables and plots, analysis	week 25	Student
Implementation results	timing (at-speed), utilization and CDC reports	week 25	Student
Diploma document	PDF (diploma-project template)	draft week 28, final week 30	Student
Defense presentation and demo	slides + live demo or video	week 30	Student

15. Assessment and Acceptance Criteria

15.1 Acceptance criteria

The project is accepted for the defense when:

- the MVP (Chapter 4) works on the AUP ZU3 and is demonstrated;
- all unit testbenches and the top-level regression pass;
- the whole system is controlled only through the JTAG pins, and every algorithm detects in hardware

exactly the faults the reference model predicts;

- BIST runs in both slow and at-speed mode, the at-speed clock meets timing, and the CDC report shows no unsafe crossings;
- the fault-coverage table is measured and every difference from the theory is explained;
- the memories work correctly in functional mode when BIST is idle;
- the design meets timing;
- code, scripts and instructions are in the repository, and the build and regression can be repeated by another person.

15.2 Assessment rubric

Table 15.1: Assessment rubric.

Criterion	Weight	Minimum (pass)	Good	Excellent
Architecture and RTL	25%	MVP IPs work	Clean, parameterised, packaged IPs; correct CDC and clock switching	+ hard-wired controller comparison, external JTAG or ICL/PDL
Verification	30%	Unit testbenches self-checking; top level passes directed tests	Reference model, constrained-random env, SVA, coverage goals met	+ full coverage closure explained, UVM version, or bugs found and documented
Experiment and analysis	20%	Coverage tables measured	Compared with theory; backgrounds, modes and diagnosis analysed	Clear insight into why each algorithm and mode fails or succeeds, with examples
Hardware integration	10%	Runs on board through the software tester	At-speed timing met, ILA validation	+ an extension (repair, MISR, 1500 wrappers)
Work process	5%	Milestones mostly met	Regular commits, weekly notes, tracker kept up to date	Independent, anticipates problems
Document and defense	10%	Complete document	Clear structure, figures, results	Publication-quality text and demo

Note

The rubric supplements, and does not replace, the official grading rules of the university.

16. Final Demonstration

1. Show the regression in the terminal (all tests PASS), the coverage report and the clean CDC report.
2. Program the AUP ZU3. The software tester resets the TAP and reads the IDCODE over JTAG.
3. In functional mode, the traffic generator uses the memories without errors.
4. Open the SIBs and switch to BIST-slow. Run March C− on all memories without faults: all pass; show the cycle count ($10n$ per memory).
5. Inject a stuck-at fault in RAM 0 and a coupling fault in the register file over JTAG. March C− fails exactly these memories, and the fail logs show the right addresses and data. MATS+ misses the coupling fault.
6. Inject an SPD fault. March SS passes in BIST-slow mode and **fails in BIST-at-speed** mode: why at-speed testing matters.
7. Load a new March program over JTAG without rebuilding the hardware, and run it.
8. Show the measured fault-coverage tables and compare them with the theory.

17. Extensions (Optional)

The following items are **not required**. They can raise the grade or become a follow-up project.

- **External JTAG:** TAP pins on a Pmod connector, driven from a PC by a USB-JTAG adapter with OpenOCD. The board then behaves like a chip on a tester.
- **ICL and PDL:** describe the network in ICL, write the tests in PDL, and generate the tester's shift sequences from them [U2].
- **IEEE 1500 wrappers** around the memory cores, accessed as instruments through the IEEE 1687 network [U15].
- **Hard-wired controller** and comparison with the programmable one (if not done as T2.8).
- **Built-in self-repair (BISR):** spare rows that replace failing addresses after BIST [B4].
- **ROM testing with a MISR**, data-retention faults with a `WAIT` instruction, port-interaction faults of the dual-port RAM.
- **More instruments on the same network,** e.g. the ECC error counters of the companion topic "ECC-Protected Sensor Data Memory on the AMD AUP ZU3".

18. Supervision, Collaboration and Risks

18.1 Supervision

- **Supervisor:** Arman Manukyan, PhD. Weekly meeting; reviews the specification, the verification plan and every milestone.
- **Code reviews:** the supervisor reviews the RTL and testbench structure at weeks 9, 13 and 19.
- **Collaboration:** git repository; questions between meetings via **[channel]**.

18.2 Risks

Table 18.1: Main risks and mitigation.

Risk	Impact	Mitigation
Scope: BIST + test access + clocks for one student	high	Simple TDR layout; external JTAG, ICL/PDL and the hard-wired controller are optional; buffer weeks 14–15
CDC and clock-switching bugs (hard to see in simulation)	high	Standard handshake and BUFGCTRL patterns [A6, U3]; SVA on handshakes; Vivado CDC report; switch only when idle
Fault behaviour in the wrapper differs from the theory	medium	Reference model first (weeks 7–9); every fault type tested in isolation
At-speed frequency not reached	medium	Pipeline the collar compare path; lower the target with the supervisor; SPD demo still works at any fast/slow ratio
Learning curve of class-based verification	medium	Warm-ups 2 and 4, [B5] early, start simple and grow the environment
Board not available	low	Everything except the final validation runs in simulation

19. Weekly Progress Tracker

Tick the box when the week's planned work is done, and write a short note (or the supervisor's initials) in the last column at the weekly meeting. Task numbers refer to Chapter 10; warm-ups to Chapter 12. Two unticked weeks in a row means the plan must be discussed and adjusted.

Table 19.1: Weekly progress tracker.

Week	Planned work (tasks)	Checkpoint result	✓	Notes / initials
Onboarding				
1	Install Vivado/Vitis, board files, repository; "hello world" on the board. Warm-up 1 (TAP state machine)	Tools run; Warm-up 1 passes	☐	
2	Warm-up 2 (RAM and FIFO); read [A2]	Warm-up 2 passes	☐	
3	Warm-up 3 (hard-coded MATS+); Warm-up 4 (first fault injection)	Warm-ups 3–4 pass	☐	
4	Warm-up 5 (bit-banged JTAG from the PS, IDCODE); read [B1] on fault models	All warm-ups accepted	☐	
Specification				
5	T1.1 memories, fault types, algorithms, expected-detection table; T1.3 instruction format, collar interface	Specification reviewed	☐	
6	T1.2 verification plan; T1.4 TAP, IEEE 1687 network, clocks and modes	Specification approved	☐	
BIST core				
7	T2.1 memories; fault wrapper (SAF, TF); T3.1 faulty-memory model	SAF/TF behave as the model	☐	
8	T2.1 coupling, address and SPD faults; four fault slots	All fault types implemented	☐	
9	T3.2 wrapper testbench with random traffic and all fault types	Wrapper verified; code review 1	☐	
10	T2.2 collar: multiplexer, address/data generators	Collar runs one element	☐	

Week	Planned work (tasks)	Checkpoint result	✓	Notes / initials
11	T2.2 comparator and fail log; T3.2 collar testbench	Collar testbench passes	☐	
12	T2.3 programmable controller; T3.1 March interpreter	Controller runs a program on collar models	☐	
13	T3.2 controller testbench + SVA	BIST core verified; code review 2	☐	
Buffer				
14	Catch up on open items; read [A1] and [A6]	Nothing behind schedule	☐	
15	Catch up; sketch the network model and the end-to-end environment	Plan for test access agreed	☐	
Test access and clocks				
16	T2.4 TAP controller (full); T2.5 SIB and TDR cells; T3.1 network model	IDCODE and SIB paths work in simulation	☐	
17	T2.5 all TDRs (clock, BIST, per-memory) with CDC handshakes; T3.2 network testbench	TDR writes/reads reach the instruments	☐	
18	T2.6 clock/mode IP and traffic generator; T3.2 clock/CDC testbench	Mode switching verified; code review 3	☐	
Integration and end-to-end verification				
19	T2.7 package IPs; block design; top-level integration	System simulates end to end through JTAG	☐	
20	T3.3 JTAG BFM, network layer, sequences, scoreboard	First end-to-end random test passes	☐	
21	T3.3 SVA and coverage; all modes and fault types; T3.4 regression script	Regression runs with several seeds	☐	
22	Coverage closure; CDC report; fix bugs; freeze RTL and testbench	Coverage goals met; CDC clean	☐	
Hardware, tester and experiment				
23	T4.1 constraints, implementation, at-speed timing closure	Bitstream with timing met	☐	
24	T4.2 software tester: JTAG, network and test layers, UART menu	BIST controlled over JTAG on the board	☐	

Week	Planned work (tasks)	Checkpoint result	✓	Notes / initials
25	T4.3 fault-coverage campaign, slow and at-speed; tables and plots	Coverage tables measured	☐	
Document and defense				
26	Write introduction, background (fault models, March tests, JTAG/IEEE 1687), architecture	Chapters to supervisor	☐	
27	Write verification and experiment chapters	Chapters to supervisor	☐	
28	Complete draft: conclusion, abstract, references	Full draft submitted	☐	
29	Revisions after feedback; defense slides	Slides draft	☐	
30	Final document; demo rehearsal	Ready for the defense	☐	

Note

Optional tasks (T2.8, T4.4) and extensions (Chapter 17) are started only when the tracker is up to date.

References and Reading List

Books

[B1] Ad J. van de Goor. *Testing Semiconductor Memories: Theory and Practice*. John Wiley & Sons, 1991.

[B2] Kenneth P. Parker. *The Boundary-Scan Handbook*. 4th ed. Springer, 2016.

[B3] Said Hamdioui. *Testing Static Random Access Memories: Defects, Fault Models and Test Patterns*. Springer, 2004.

[B4] Laung-Terng Wang, Cheng-Wen Wu, and Xiaoqing Wen. *VLSI Test Principles and Architectures: Design for Testability*. Morgan Kaufmann, 2006.

[B5] Chris Spear and Greg Tumbush. *SystemVerilog for Verification: A Guide to Learning the Testbench Language Features*. 3rd ed. Springer, 2012.

[B6] Eduard Cerny, Surrendra Dudani, John Havlicek, and Dmitry Korchemny. *SVA: The Power of Assertions in SystemVerilog*. 2nd ed. Springer, 2015.

[B7] Ray Salemi. *The UVM Primer: A Step-by-Step Introduction to the Universal Verification Methodology*. Boston Light Press, 2013.

[B8] Sarah L. Harris and David Money Harris. *Digital Design and Computer Architecture: ARM Edition*. Morgan Kaufmann, 2016.

[B9] Michael L. Bushnell and Vishwani D. Agrawal. *Essentials of Electronic Testing for Digital, Memory and Mixed-Signal VLSI Circuits*. Springer, 2000.

[B10] Stuart Sutherland, Simon Davidmann, and Peter Flake. *SystemVerilog for Design: A Guide to Using SystemVerilog for Hardware Design and Modeling*. 2nd ed. Springer, 2006.

Articles and papers

[A1] Jeff Rearick et al. “IJTAG (Internal JTAG): A Step Toward a DFT Standard”. In: *Proceedings of the IEEE International Test Conference (ITC)*. 2005.

[A2] Ad J. van de Goor. “Using March Tests to Test SRAMs”. In: *IEEE Design & Test of Computers* 10.1 (1993), pp. 8–14.

[A3] D. S. Suk and S. M. Reddy. “A March Test for Functional Faults in Semiconductor Random Access Memories”. In: *IEEE Transactions on Computers* C-30.12 (1981), pp. 982–985.

[A4] Said Hamdioui, Ad J. van de Goor, and Mike Rodgers. “March SS: A Test for All Static Simple RAM Faults”. In: *Proceedings of the IEEE International Workshop on Memory Technology, Design and Testing (MTDT)*. 2002, pp. 95–100.

[A5] Yervant Zorian. “A Distributed BIST Control Scheme for Complex VLSI Devices”. In: *Proceedings of the 11th IEEE VLSI Test Symposium (VTS)*. 1993, pp. 4–9.

[A6] Clifford E. Cummings. “Clock Domain Crossing (CDC) Design & Verification Techniques Using SystemVerilog”. In: *Synopsys Users Group Conference (SNUG), Boston*. 2008.

User guides, datasheets and standards

[U1] IEEE. *IEEE Standard for Test Access Port and Boundary-Scan Architecture (IEEE Std 1149.1-2013)*. IEEE. 2013.

[U2] IEEE. *IEEE Standard for Access and Control of Instrumentation Embedded within a Semiconductor Device (IEEE Std 1687-2014)*. IEEE. 2014.

[U3] AMD. *UltraScale Architecture Clocking Resources User Guide (UG572)*. Advanced Micro Devices.

[U4] AMD. *UltraScale Architecture Memory Resources User Guide (UG573)*. Advanced Micro Devices.

[U5] AMD. *UltraScale Architecture Configurable Logic Block User Guide (UG574)*. Advanced Micro Devices.

[U6] AMD. *Vivado Design Suite User Guide: Creating and Packaging Custom IP (UG1118)*. Advanced Micro Devices.

[U7] AMD. *Vivado Design Suite User Guide: Designing IP Subsystems Using IP Integrator (UG994)*. Advanced Micro Devices.

[U8] AMD. *Clocking Wizard LogiCORE IP Product Guide (PG065)*. Advanced Micro Devices.

[U9] AMD. *Vivado Design Suite User Guide: Design Analysis and Closure Techniques (UG906)*. Advanced Micro Devices.

[U10] AMD. *Zynq UltraScale+ Device Technical Reference Manual (UG1085)*. Advanced Micro Devices.

[U11] AMD. *AXI GPIO LogiCORE IP Product Guide (PG144)*. Advanced Micro Devices.

[U12] AMD. *Vitis Unified Software Platform Documentation: Embedded Software Development (UG1400)*. Advanced Micro Devices.

[U13] AMD. *Integrated Logic Analyzer LogiCORE IP Product Guide (PG172)*. Advanced Micro Devices.

[U14] AMD. *Vivado Design Suite User Guide: Logic Simulation (UG900)*. Advanced Micro Devices.

[U15] IEEE. *IEEE Standard Testability Method for Embedded Core-based Integrated Circuits (IEEE Std 1500-2005)*. IEEE. 2005.

[U16] IEEE. *IEEE Standard for SystemVerilog—Unified Hardware Design, Specification, and Verification Language (IEEE Std 1800-2017)*. IEEE. 2018.

List of Abbreviations

BIST Built-In Self-Test.

MBIST Memory Built-In Self-Test.

BISR Built-In Self-Repair.

DfT Design for Testability.

MUT Memory Under Test.

SAF Stuck-At Fault.

TF Transition Fault.

CF Coupling Fault (CFin: inversion, CFid: idempotent, CFst: state).

AF Address-Decoder Fault.

DRF Data-Retention Fault.

MISR Multiple-Input Signature Register.

FPGA Field-Programmable Gate Array.

PL Programmable Logic.

PS Processing System.

MPSoC Multiprocessor System-on-Chip.

RTL Register-Transfer Level.

BRAM Block RAM.

LUTRAM distributed RAM built from look-up tables.

AXI Advanced eXtensible Interface.

FSM Finite-State Machine.

SVA SystemVerilog Assertions.

UVM Universal Verification Methodology.

DUT Device Under Test.

ILA Integrated Logic Analyzer.

UART Universal Asynchronous Receiver-Transmitter.

MVP Minimum Viable Product.

JTAG Joint Test Action Group (IEEE 1149.1 test access port).

TAP Test Access Port.

IJTAG Internal JTAG (IEEE 1687).

TDR Test Data Register.

SIB Segment Insertion Bit.

ICL Instrument Connectivity Language (IEEE 1687).

PDL Procedural Description Language (IEEE 1687).

CDC Clock-Domain Crossing.

MMCM Mixed-Mode Clock Manager.

GPIO General-Purpose Input/Output.

EMIO Extended Multiplexed I/O (PS peripherals routed to the PL).

Glossary

March test memory test made of a sequence of March elements; each element visits every address in a given order and applies the same short list of read/write operations.

collar (memory wrapper) logic around one memory that takes control of it in test mode, applies the operations, compares read data and logs failures.

data background the data pattern written as “0” in a word-oriented memory (e.g. all zeros, checkerboard 0101…); several backgrounds are needed to detect faults between bits of the same word.

aggressor / victim in a coupling fault, the cell whose access or state disturbs (aggressor) and the cell that is disturbed (victim).

fault coverage percentage of injected faults of a given type that an algorithm detects.

fail log record of each detected mismatch: memory, address, expected and actual data, March element and operation.

functional coverage measurement of which scenarios, values and corner cases the tests have actually exercised.

scoreboard testbench component that compares the DUT output with the output of a reference model.

at-speed test running the test at the memory’s real operating clock, so that faults that appear only at high speed (slow cells, slow paths) are detected.

active scan path the chain of registers currently between TDI and TDO; in IEEE 1687 it changes when SIBs are opened or closed.

instrument in IEEE 1687, any on-chip block accessed through the network: here the BIST controller, the collars, the fault wrappers and the clock/mode controller.